

1. Overview

This code implements a **Convolutional Neural Network (CNN)** to classify images from the **CIFAR-10 dataset** using **PyTorch**.
It includes:

- A custom CNN model (**SimpleCNN**)
- Data preprocessing and augmentation pipeline
- Training, validation, and testing routines
- Model checkpointing and learning rate scheduling

The CIFAR-10 dataset contains **60,000 color images (32×32)** across **10 classes** such as airplane, car, bird, cat, etc.

2. Model Architecture: **SimpleCNN**

2.1 Architecture Summary

The model defines **three convolutional blocks** followed by a **fully connected classifier**:

Block	Layers	Output Size	Description
Conv Block 1	2 × Conv(3→64) + BN + ReLU + MaxPool	16×16	Extracts low-level features (edges, textures)
Conv Block 2	2 × Conv(64→128) + BN + ReLU + MaxPool	8×8	Captures mid-level features
Conv Block 3	2 × Conv(128→256) + BN + ReLU + MaxPool	4×4	Learns high-level spatial features
Classifier	Flatten → FC(4096→512) → ReLU → Dropout(0.3) → FC(512→10)	10	Outputs class logits

2.2 Design Highlights

- **Batch Normalization (BN):** stabilizes training and accelerates convergence.
- **ReLU Activation:** adds non-linearity for complex representations.
- **Dropout (0.3):** regularization to prevent overfitting.
- **MaxPooling:** progressively reduces spatial resolution while retaining essential features.

2.3 Forward Pass

```
def forward(self, x):  
    x = self.features(x)  
    x = self.classifier(x)  
    return x
```

Each image passes sequentially through convolutional layers, activations, pooling, and finally fully connected layers for classification.

3. Dataset and Preprocessing

The dataset logic resides in `utils.py`.

3.1 Transformations

The code uses **data augmentation** for robustness:

Training Transformations

```
RandomCrop(32, padding=4)  
RandomHorizontalFlip()  
ColorJitter(0.2, 0.2, 0.2, 0.2)  
RandomRotation(15)  
ToTensor()  
Normalize(mean=CIFAR_MEAN, std=CIFAR_STD)
```

These increase generalization by simulating image variations.

Validation/Test Transformations

Only normalization and tensor conversion are applied (no augmentation).

3.2 Dataloader Function

```
train_loader, val_loader, test_loader = get_dataloaders(batch_size=128)
```

- The training set is split into **train (45,000)** and **validation (5,000)** samples.
 - **Parallel loading** is enabled (`num_workers=7`) for efficiency.
 - `random_split()` ensures a reproducible split using a fixed random seed.
-

4. Training Pipeline

4.1 Training Loop (`train_one_epoch`)

Performs a single epoch of training:

1. Puts the model in `train()` mode.
 2. Iterates through the batches.
 3. Computes **loss** via `CrossEntropyLoss(label_smoothing=0.1)`.
 4. Performs **backpropagation** (`loss.backward()`).
 5. Updates weights with **AdamW optimizer**.
 6. Tracks average **loss** and **accuracy**.
-

4.2 Validation (`eval_model`)

Similar to training, but:

- Model runs in `eval()` mode (disables dropout & batchnorm updates).
- Wrapped in `torch.no_grad()` to save memory and computation.
- No weight updates.

4.3 Optimization Setup

Component	Choice	Explanation
Loss	<code>CrossEntropyLoss(label_smoothing=0.1)</code>	Adds small uniform noise to labels, improves calibration.
Optimizer	<code>AdamW</code>	Adam with decoupled weight decay (better generalization).
Learning Rate Scheduler	<code>CosineAnnealingLR</code>	Smoothly decays LR following a cosine curve.

5. Checkpointing and Model Saving

Whenever validation accuracy improves:

- The model's state is saved as `best_cnn_cifar10.pth`.
- Additionally, a detailed checkpoint `best_cnn_cifar10.ckpt` is saved containing:
 - Architecture name
 - Epoch number
 - Best validation accuracy
 - Model weights
 - Normalization stats (mean, std)

To prevent **MPS memory spikes on macOS**, the model is temporarily moved to CPU before saving.

6. Testing and Final Evaluation

After all epochs:

- The best checkpoint is reloaded.
- The model is evaluated on the **test set**.
- Final accuracy and loss are printed.

Example output:

Epoch 021	Train acc 0.8591	loss 0.8733	Val acc 0.8326	loss 0.9005
Epoch 022	Train acc 0.8634	loss 0.8652	Val acc 0.8232	loss 0.9280
Epoch 023	Train acc 0.8697	loss 0.8526	Val acc 0.8278	loss 0.9207
Epoch 024	Train acc 0.8736	loss 0.8445	Val acc 0.8484	loss 0.8652
Epoch 025	Train acc 0.8790	loss 0.8330	Val acc 0.8478	loss 0.8800
Epoch 026	Train acc 0.8829	loss 0.8250	Val acc 0.8572	loss 0.8493
13%				

7. Key Design Choices and Rationale

Aspect	Choice	Reason
SimpleCNN	Custom lightweight model	Easier to train on local GPU/CPU (no heavy ResNet dependency).
Label smoothing	0.1	Improves generalization and prevents overconfidence.
AdamW + CosineAnnealingLR		Helps stabilize training and adapt learning rate over epochs.
Augmentations	Rotation, jitter, flip	Improve robustness to variations in data.
Checkpointing	On best validation accuracy	Prevents overfitting and preserves best model state.

8. Summary

This code demonstrates a **complete deep learning pipeline** for image classification:

- From **data preprocessing** to **model design, training, evaluation**, and **saving**.
 - It uses **modern PyTorch practices** — modular functions, label smoothing, AdamW, cosine annealing, and checkpointing.
 - The implementation is efficient, well-structured, and portable for experiments with different architectures or datasets.
-